OWL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

OWL

https://github.com/mikel-egana-aranguren/ABD

CC BY

OWL

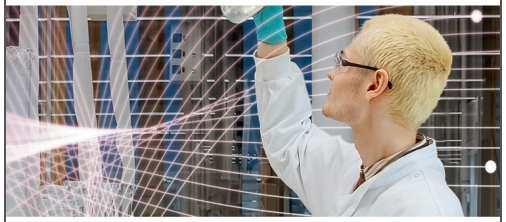
the LEGO Group

Lead Ontology Engineer - Semantic Data Modelling

- Proven expertise in Semantic Modelling and Ontology Design using open standards.
- Proficiency in RDF, OWL, and SHACL for developing and managing ontologies.
- Experience in data governance with knowledge graphs and semantic models.
- Strong programming skills in languages such as Python and TypeScript.
- Excellent communication and collaboration skills to work effectively within a multidisciplinary team.
- Understanding Domain Driven Design, loosely coupled components, and event driven architecture.
- Travel 15-20 days a year (dependent on need).
- Communicate effortlessly in English both written and verbally.

OWL

AstraZeneca Unirse a la Red de Talento



← Todos los Trabajos

Ontologist

Macclesfield, , United Kingdom

Aplicar Ahora | Añadir al Carro de Trabajo

Descripción del puesto vacante

Introduction to role

Join our Global Operations Data Office as an Ontologist and be part of a team that is transforming the pharmaceutical industry through data innovation.

Our mission is to ensure AI-ready data that supports our Operations 2030 ambition, enabling life-changing medicines. We partner with various business areas, including Pharmaceutical Technology and Development, Manufacturing, and Global Supply Chain, operating from hub sites in the UK, Sweden, US, China, and Global Technology Centres in India and Mexico. Here, your work will directly impact patients by empowering the business to perform at its peak through cutting-edge science and digital technology platforms.

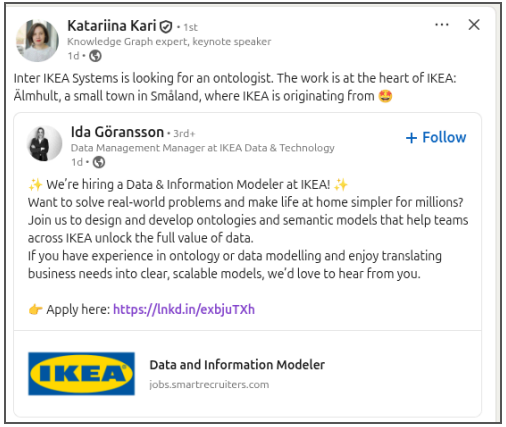
Accountabilities

As an Ontologist, you will play a crucial role in developing semantic data solutions across manufacturing, supply chain, and development functions. You will engineer robust ontology driven knowledge models, integrate structured and unstructured data, and apply semantic standards to enhance data interoperability. Collaborating closely with product teams and data engineers, you will bring semantic capabilities into real-world business applications. This is your chance to work at the forefront of semantic data technologies in the pharmaceutical industry—enabling FAIR data, accelerating AI use cases, and driving smarter digital operations.

Essential Skills/Experience

- Proficiency with RDF, SKOS, OWL, SHACL, SPARQL, and other semantic web technologies.

OWL


Katariina Kari • 1st
Knowledge Graph expert, keynote speaker
1d •
Inter IKEA Systems is looking for an ontologist. The work is at the heart of IKEA: Älmhult, a small town in Småland, where IKEA is originating from 🤩
Ida Göransson • 3rd+
Data Management Manager at IKEA Data & Technology
1d •
+ Follow
✨ We're hiring a Data & Information Modeler at IKEA! ✨
Want to solve real-world problems and make life at home simpler for millions?
Join us to design and develop ontologies and semantic models that help teams across IKEA unlock the full value of data.
If you have experience in ontology or data modelling and enjoy translating business needs into clear, scalable models, we'd love to hear from you.
👉 Apply here: https://lnkd.in/exbjuTXh
IKEA
Data and Information Modeler
jobs.smartrecruiters.com

OWL

OWL: Web Ontology Language

Estándar oficial del W3C para crear ontologías en la web con un semántica precisa y formal

OWL

Se basa en Lógica Descriptiva (DL) para representar computacionalmente un dominio de conocimiento:

- Razonamiento automático: inferir conocimiento "nuevo" (*), consultas, consistencia, clasificar entidades contra la ontología, ...
- Integrar información dispersa mediante un vocabulario común

OWL

No es un lenguaje esquema con restricciones, se basa en inferencia (Para eso esta SHACL)

El mismo lenguaje RDF para definir datos y su vocabulario* (NoSQL!RDF!)

Sintaxis RDF/XML

```
<owl:Class rdf:about="#arm">
   <rdfs:subClassOf>
    <owl:Restriction>
         <owl:onProperty rdf:resource="#part_of"/>
         <owl:someValuesFrom rdf:resource="#body"/>
    </owl:Restriction>
   </rdfs:subClassOf>
</owl:Class>
```

Manchester OWL Syntax

Manchester OWL Syntax: arm subClassOf art_of some body

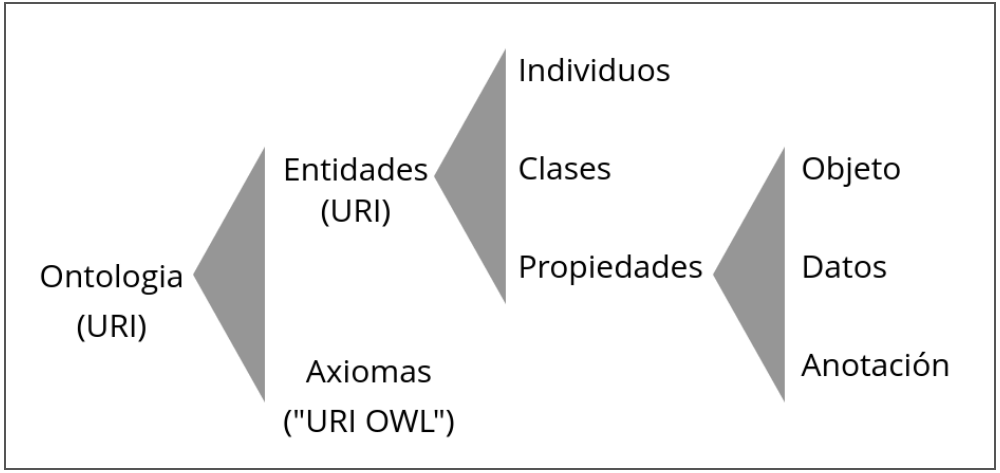
Semántica OWL

Entidades: las entidades del dominio de conocimiento, identificadas con URIs, introducidas por el desarrollador ("Mikel", "participa_en", ...)

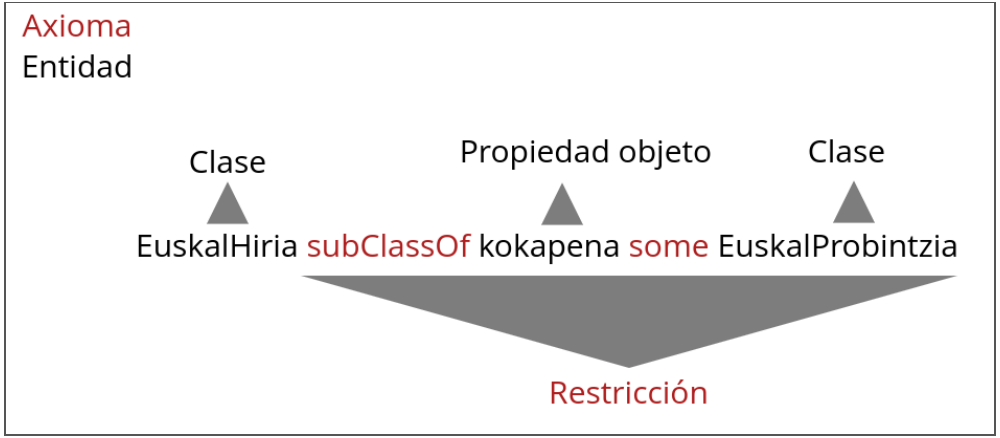
Axiomas: relacionan las entidades mediante el vocabulario lógico que ofrece OWL (namespace OWL)

Una ontología puede importar otra (owl:import) y hacer referencia a sus entidades mediante axiomas

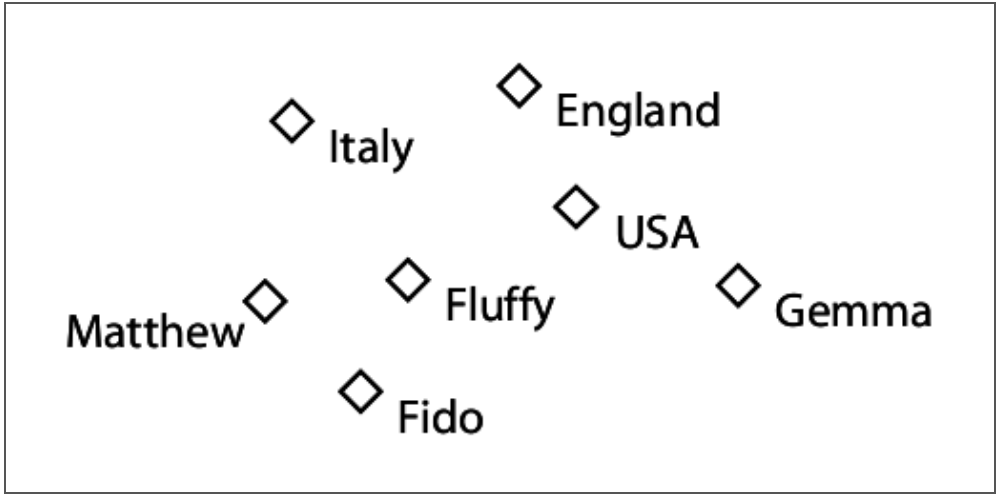
Semántica OWL



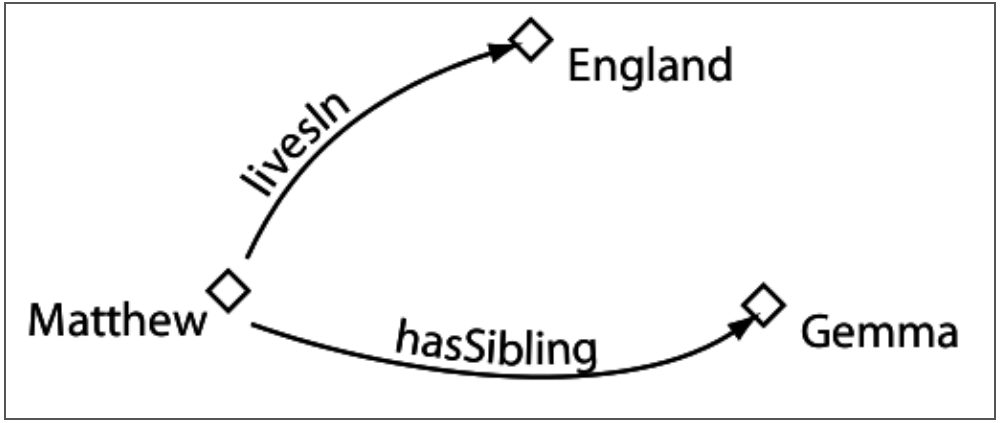
Semántica OWL



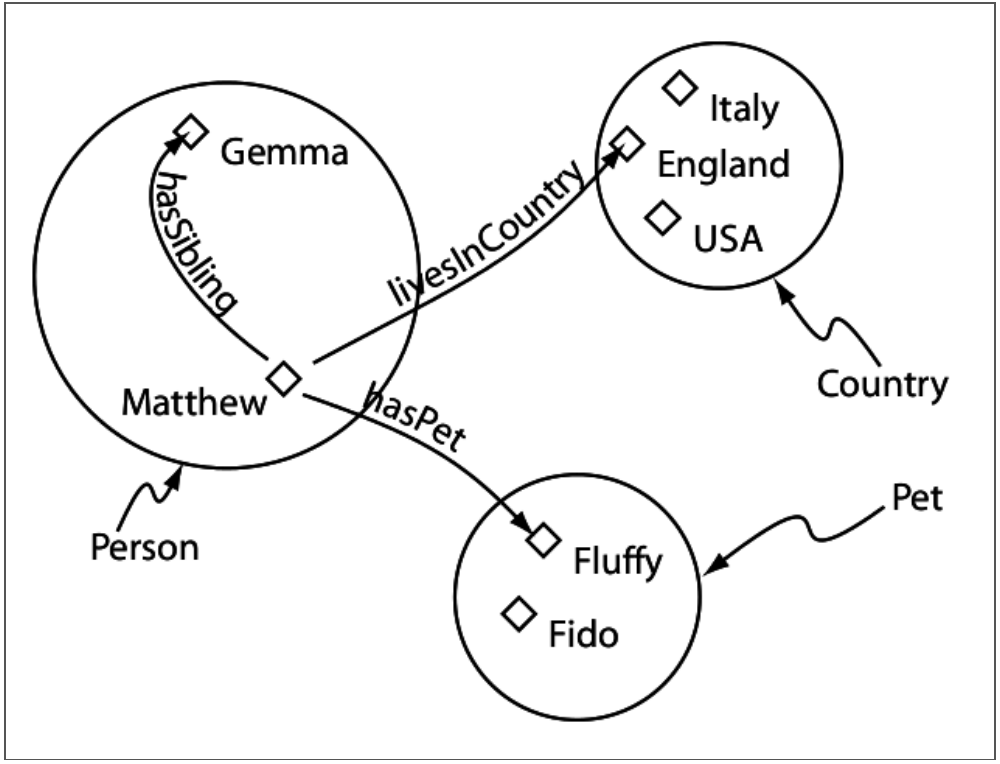
Individuos



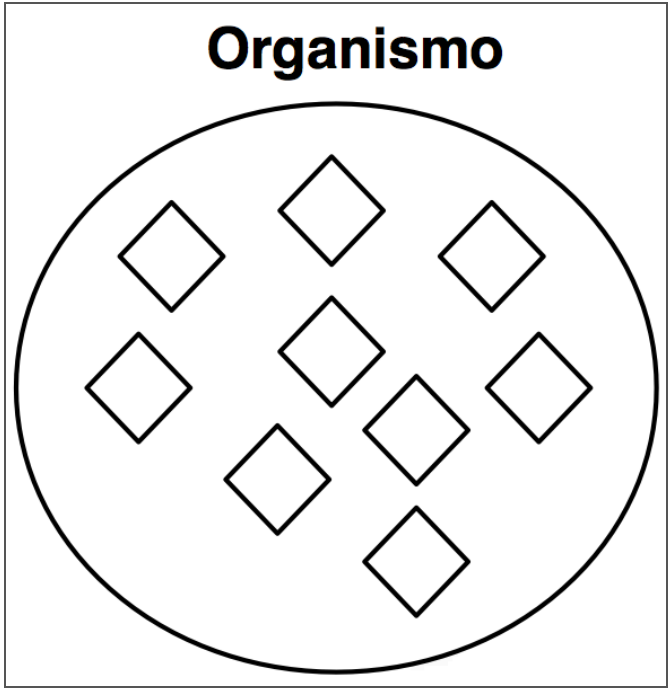
Propiedades



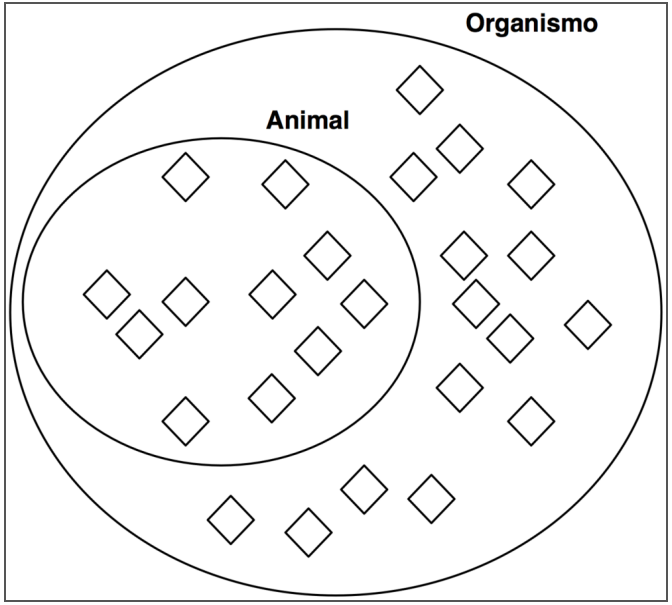
Clases


Gemma
hasSibling
livesInCountry
Italy
England
USA
Country
Matthew
hasPet
Person
Pet
Fluffy
Fido

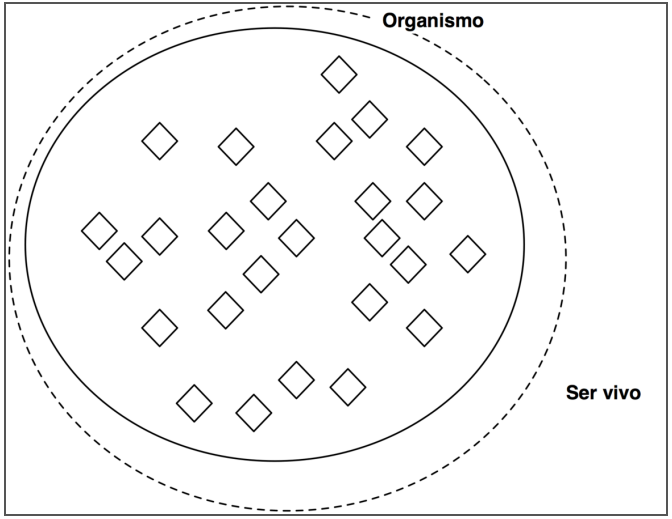
Clases



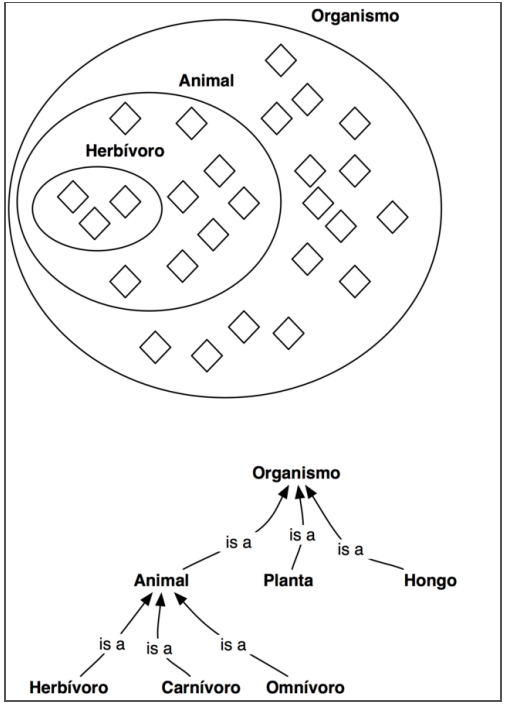
Clases subclase



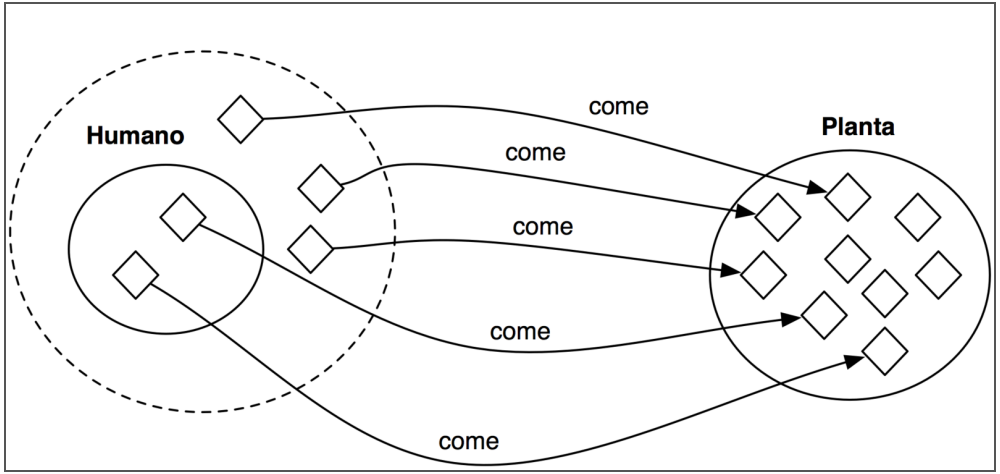
Clases equivalentes



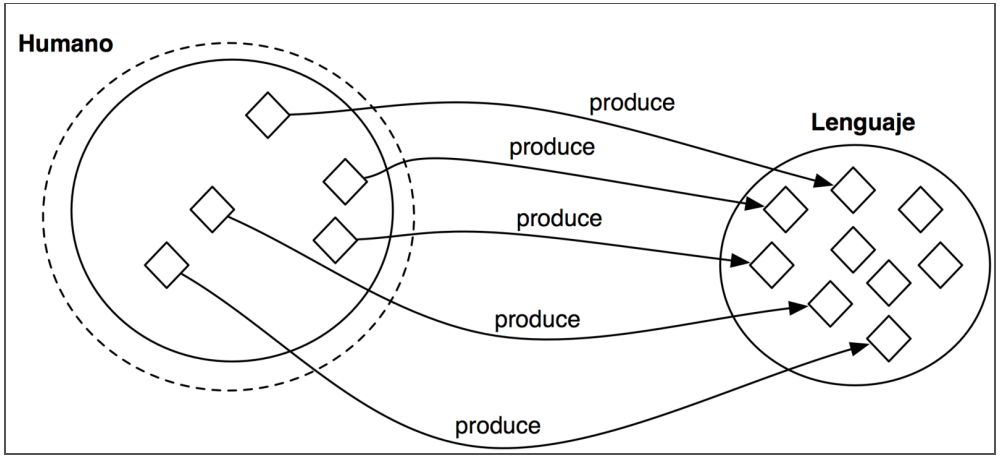
Jerarquía de clases (Taxonomía)



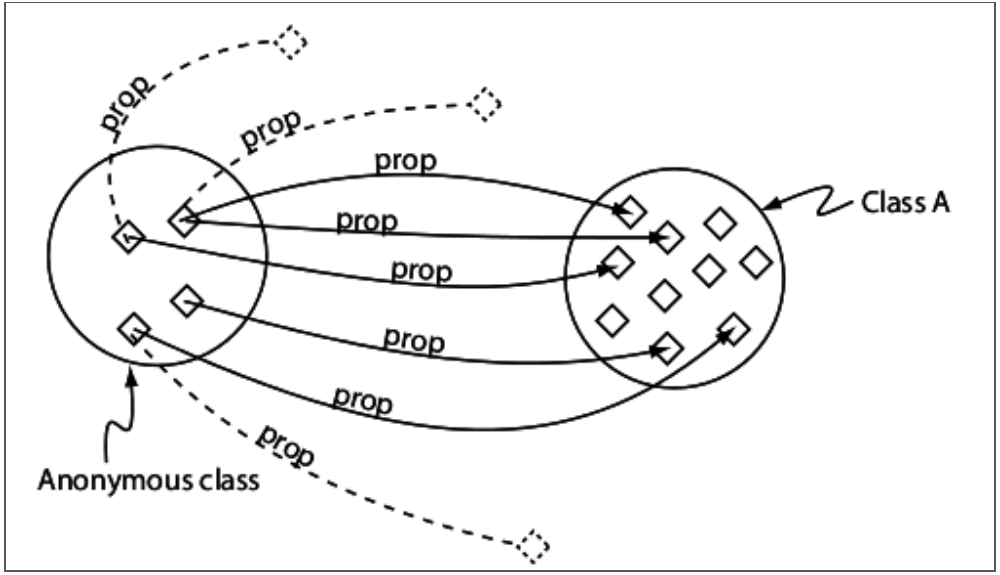
Condiciones necesarias



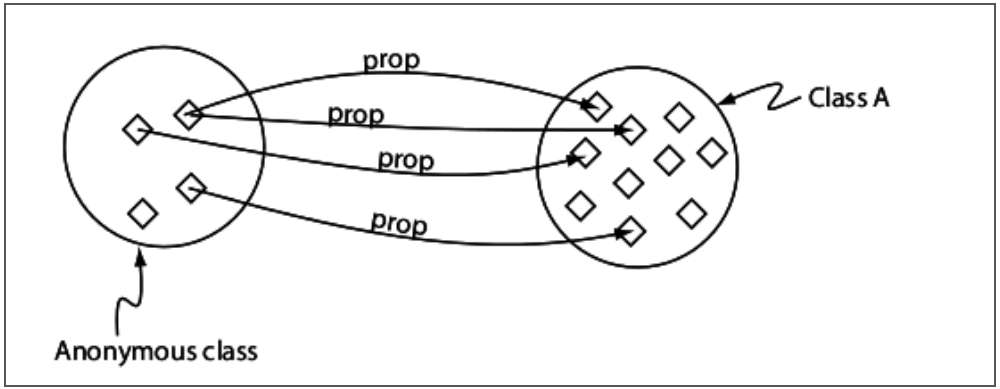
Condiciones necesarias y suficientes



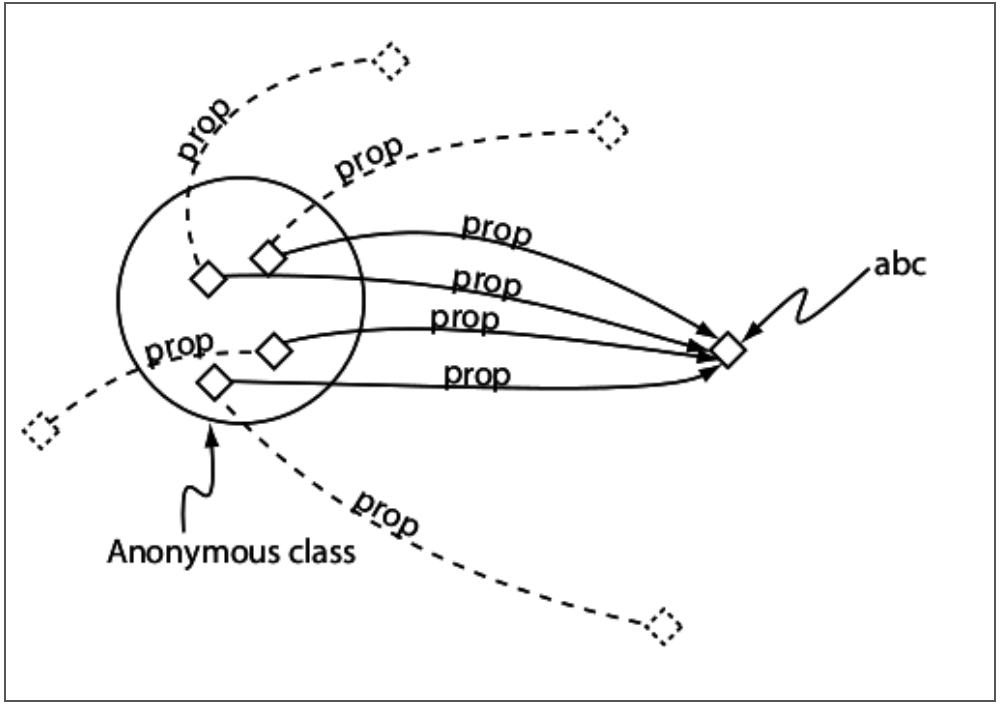
Restricción existencial



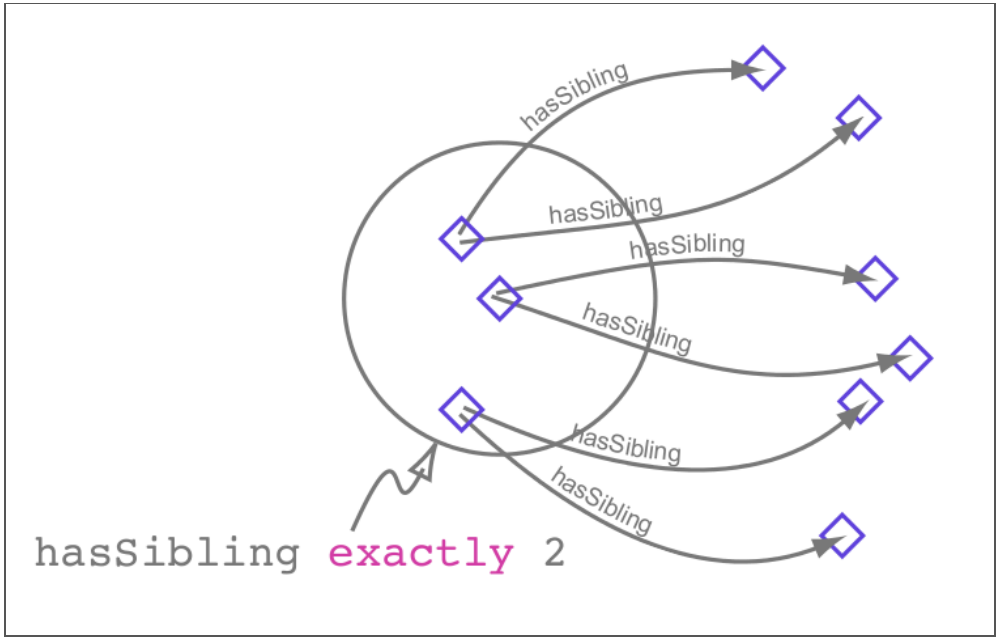
Restricción universal



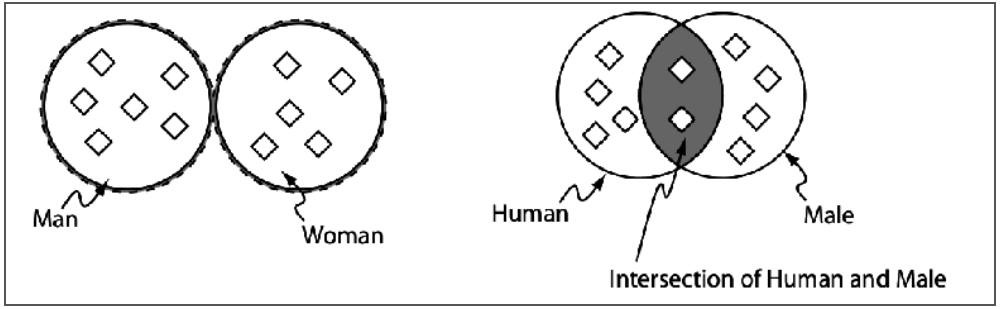
Restricción un individuo (value)



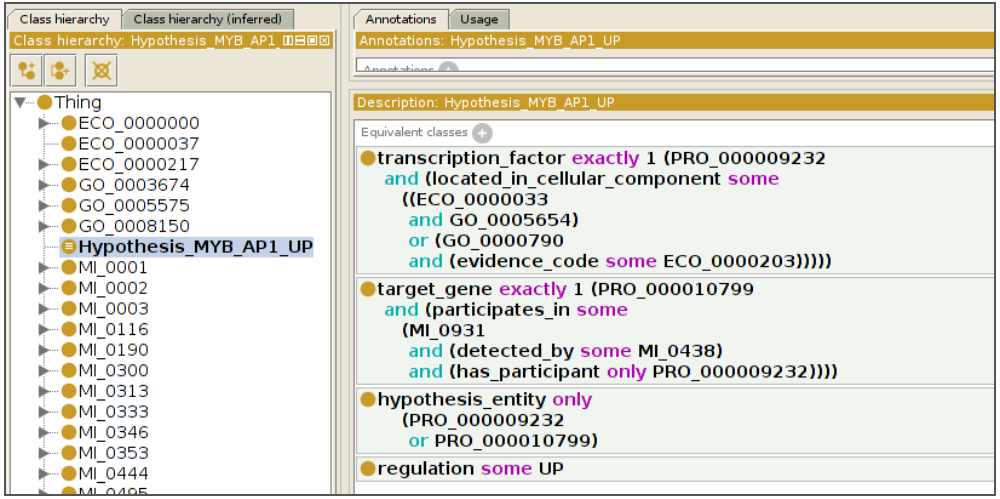
Restricciones cardinales



disjointFrom, not, or, and



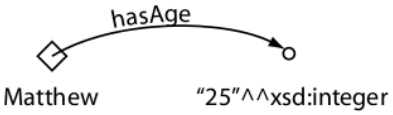
Expresiones complejas



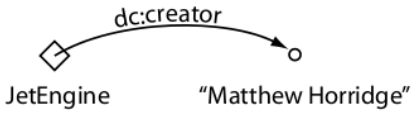
Propiedades

Propiedades objeto

Propiedades datos

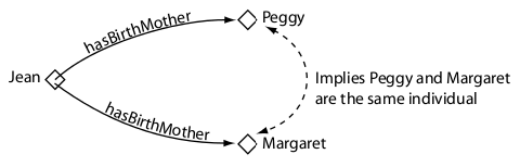


Propiedades anotacion

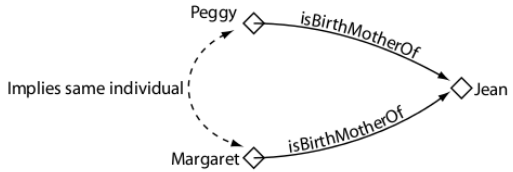


Propiedades

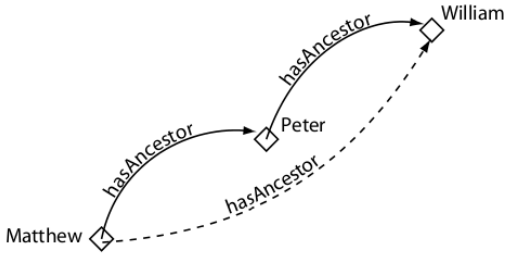
Funcional



Inversa funcional

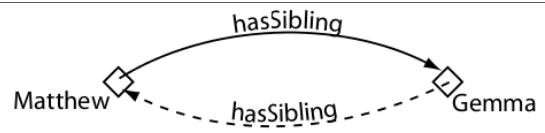


Transitiva

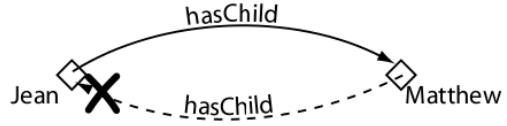


Propiedades

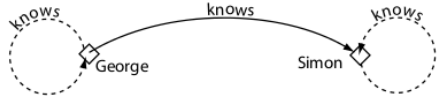
Simetrica



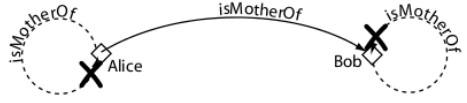
Antisimetrica*



Reflexiva

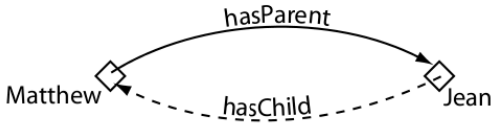


Irreflexiva*

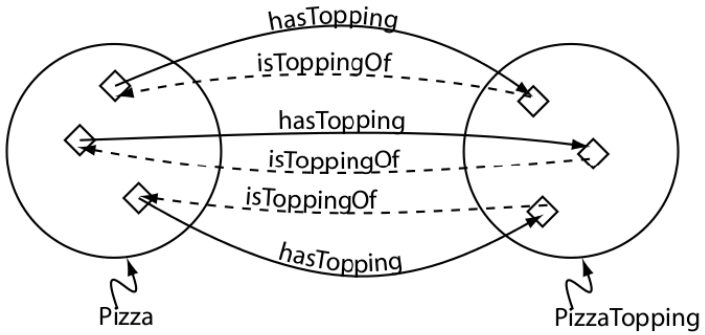


Propiedades

Propiedades inversas



Dominio y rango



Individuos

Miembro de una o más clases: rdf:type (RDF! Web Semantica! NoSQL!)

Igual (owl:sameas) o diferente (owl:differentfrom) a otro individuo

Relaciones binarias con otros individuos o datos (triples), positivas o negativas

Razonamiento automático

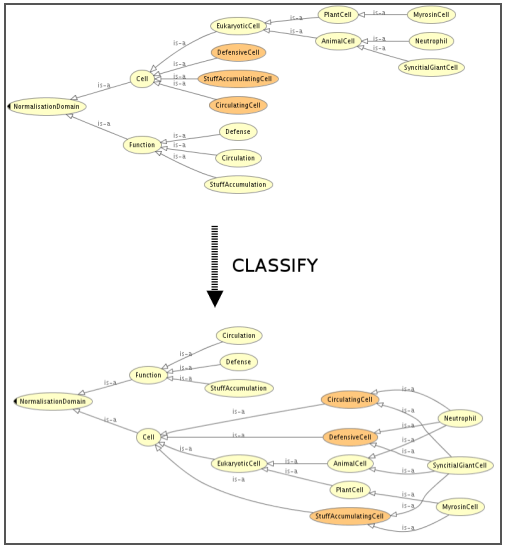
Un razonador infiere los "nuevos" axiomas que implican los axiomas que hemos introducido en la ontología

El razonador infiere todos los axiomas; es útil para tratar con conocimiento complejo

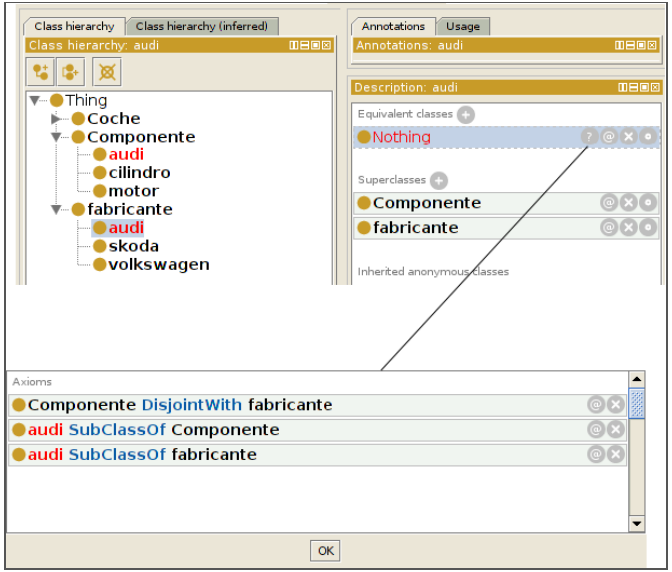
Open World Assumption

No Unique Name Assumption

Razonamiento automático: mantener taxonomía



Razonamiento automático: Consistencia

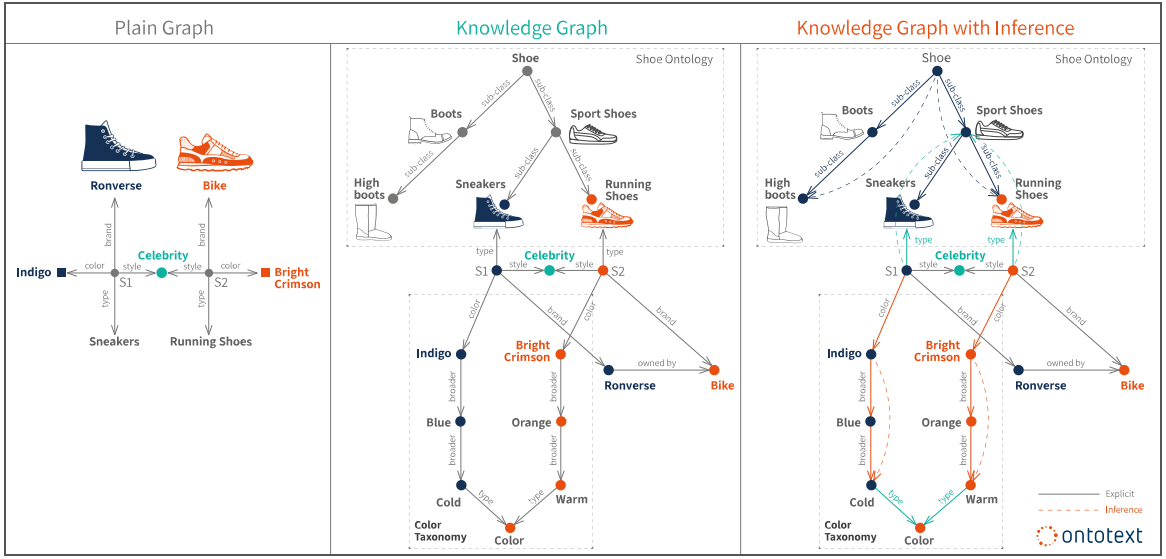


Razonamiento automático: Clasificar

Clasificar entidades: dada una entidad nueva, como se relaciona con las demas entidades (types, equivalentTo, subClassOf, triples)

Una consulta es una clase anónima que clasificamos contra la ontología como si fuese una entidad

Knowledge Graphs



Knowledge Graphs

WikiData: https://www.wikidata.org/

DBPedia: https://www.dbpedia.org/about/

Uniprot: https://sparql.uniprot.org/

...

OWL 2: Perfiles

OWL 2 define sublenguajes con diferentes trade-offs expresividad/complejidad:

OWL 2 EL: para ontologías grandes (millones de clases). Razonamiento en tiempo polinomial

OWL 2 QL: optimizado para consultas sobre datos. Reescritura de queries

OWL 2 RL: implementable con motores de reglas. Razonamiento eficiente

OWL 2 EL

Usado en ontologías biomédicas: SNOMED CT (>300,000 clases)

Soporta: SubClassOf, equivalentClass, existencial (some)

No soporta: universal (only), negación, disyunción

Complejidad: PTime para razonamiento de subsunción

Razonador: ELK

OWL 2 QL

Basado en DL-Lite: query rewriting sobre bases de datos relacionales

Permite consultar BBDD relacionales virtualizadas como RDF

No soporta: existencial en lado derecho, restricciones de cardinalidad

Complejidad: LogSpace para razonamiento

Herramienta: Ontop (OBDA)

OWL 2 RL

Subset implementable con reglas (forward chaining)

Compatible con sistemas de producción basados en reglas

Soporta la mayoría de constructores OWL pero con restricciones

Complejidad: PTime

GraphDB usa reglas para implementar razonamiento OWL-Horst (similar a RL)

Lógica Descriptiva: DL

OWL se basa en familias de lógicas descriptivas

DL: fragmentos decidibles de lógica de primer orden

Nomenclatura: **SROIQ(D)** - OWL 2 DL

- S: extensión de ALC con transitividad
- R: propiedades complejas, reflexividad
- O: nominals (individuos en expresiones de clase)
- I: propiedades inversas

Razonadores OWL

OWL API: librería Java para manipular ontologías

HermiT: razonador basado en hypertableau (OWL 2 DL)

Pellet: razonador comercial de Stardog

FaCT++: razonador C++, rápido

ELK: razonador OWL 2 EL ultra-rápido

Protégé

Editor de ontologías OWL de Stanford: https://protege.stanford.edu/

Interfaz gráfica para crear clases, propiedades, axiomas

Integración con razonadores (HermiT, Pellet, ELK, ...)

Plugins: visualización, validación, documentación

Exportación múltiples formatos: RDF/XML, Turtle, Manchester, OWL/XML

Patrones de diseño OWL

Value Partition: clasificar por valores de una propiedad

Defined Classes: clases definidas completamente (necessary & sufficient)

Closure Axiom: cerrar el mundo para propiedades específicas

N-ary Relations: representar relaciones complejas

Ontology Design Patterns

Value Partition - Ejemplo

```
Class: Size
    SubClassOf: owl:Thing
    DisjointUnionOf: Small, Medium, Large

Class: TShirt
    SubClassOf: hasSize some Size

Class: LargeTShirt
```

Open World Assumption (OWA)

La ausencia de información no implica falsedad

Si no sabemos que Mikel vive en Bilbao, no asumimos que vive en otro sitio

Diferente de bases de datos relacionales (Closed World)

Consecuencia: las restricciones universales no son restrictivas por defecto

Para "cerrar" el mundo: usar propiedades funcionales, closure axioms, SHACL

No Unique Name Assumption (NUNA)

Dos URIs diferentes pueden referirse a la misma entidad

ex:Bilbao y ex:Bilbo pueden ser owl:sameAs

Útil para integración de datos de diferentes fuentes

Permite reconciliación automática mediante razonamiento

owl:differentFrom para forzar distinción

Anotaciones OWL

Metadatos sobre la ontología y sus entidades:

rdfs:label, rdfs:comment: etiquetas y descripciones

dc:creator, dc:date: Dublin Core para autoría

owl:versionInfo, owl:priorVersion: versionado

rdfs:seeAlso, rdfs:isDefinedBy: referencias

Pueden ser multilingües con language tags

Módulos OWL

owl:imports: importar otra ontología completa

Modularización: dividir ontologías grandes en módulos

Reutilización: importar vocabularios estándar (FOAF, DC, Schema.org)

Problema: importar todas las axiomas transitivamente

Solución: MIREOT (extracción selectiva de términos)

OWL y SPARQL

SPARQL consulta el grafo RDF serializado

Por defecto no considera axiomas inferidos

GraphDB: materializa inferencias o razona en tiempo de consulta

SPARQL 1.1: entailment regimes (RDF, RDFS, OWL)

Federated queries: consultar múltiples endpoints con ontologías

Inconsistencias en OWL

Ontología inconsistente: contiene contradicciones lógicas

Clase insatisfacible: no puede tener instancias (equivalente a owl:Nothing)

Causas comunes:

- Clases disjuntas con instancias compartidas
- Restricciones de cardinalidad contradictorias
- Propiedades funcionales con múltiples valores

Explicaciones de inferencias

Justification: conjunto mínimo de axiomas que causa una inferencia

Herramientas: Protégé plugin de explicación

Debugging: identificar axiomas problemáticos

Puede haber múltiples justificaciones para una inferencia

Punning en OWL 2

Permitir que la misma URI sea clase, propiedad e individuo simultáneamente

Útil para metamodelado: clases que son instancias de otras clases

Ejemplo: ex:Mammal rdf:type owl:Class y ex:Mammal rdf:type ex:BiologicalCategory

Semántica: tratadas como entidades separadas según contexto

Property Chains

Definir propiedades complejas mediante composición

```
ObjectPropertyChain:
    hasPadre o hasPadre → hasAbuelo


ObjectPropertyChain:
    localizadoEn o localizadoEn → localizadoEn
    (transitividad indirecta)
```

Keys en OWL 2

Identificar individuos únicamente mediante combinación de propiedades

```
Class: Person
    HasKey: hasSSN


Class: Publication
    HasKey: hasISBN, hasEdition
```

Similar a claves primarias en bases de datos relacionales

Ontologías de alto nivel (Upper Ontologies)

Ontologías genéricas para cualquier dominio:

BFO (Basic Formal Ontology): ciencias biomédicas

SUMO (Suggested Upper Merged Ontology)

DOLCE: descriptiva, para integración

Proveen categorías básicas: Objeto, Proceso, Cualidad, Rol…

OWL en la industria: Casos de uso

Farmacéuticas: descubrimiento de fármacos, modelado de efectos adversos

Finanzas: compliance, risk management, FIBO (Financial Industry Business Ontology)

Manufactura: PLM (Product Lifecycle Management), IoT

E-commerce: catálogos de productos, recomendaciones semánticas

Salud: terminologías médicas (SNOMED CT), historiales clínicos

Metodologías para construir ontologías

METHONTOLOGY: metodología clásica de desarrollo

NeOn: escenarios de reutilización y colaboración

Agile: desarrollo iterativo, validación continua

Competency Questions: requisitos funcionales mediante preguntas

Mejores prácticas: modularización, versionado, documentación, testing

Testing de ontologías

Unit tests: verificar axiomas específicos se infieren correctamente

Consistency checking: no inconsistencias ni clases insatisfacibles

Competency question tests: responder queries esperadas

Performance tests: tiempo de razonamiento aceptable

Herramientas: Tawny-OWL, ROBOT

Métricas de calidad de ontologías

Completitud: cobertura del dominio

Concisión: sin redundancias o axiomas derivables

Coherencia: sin contradicciones lógicas

Modularidad: organización en componentes reutilizables

Anotaciones: documentación adecuada (labels, comments)

Herramienta: OOPS! (Ontology Pitfall Scanner)

OWL y Machine Learning

Knowledge Graph Embeddings: representar entidades en espacio vectorial

Neuro-symbolic AI: combinar razonamiento simbólico con redes neuronales

Explicabilidad: ontologías proporcionan contexto interpretable

Transfer learning: conocimiento previo codificado en ontologías

Ejemplo: OWL2Vec

Recursos para aprender OWL

OWL 2 Web Ontology Language Primer

Pizza Tutorial (Protégé)

OWL 2 Profiles Tutorial

A Practical Guide To Building OWL Ontologies

OBO Foundry: ontologías biomédicas de alta calidad